

Article

A Deep Learning Algorithm for the Max-Cut Problem Based on Pointer Network Structure with Supervised Learning and Reinforcement Learning Strategies

Shenshen Gu ^{*,†} and Yue Yang

School of Mechatronic Engineering and Automation, Shanghai University, Shanghai 200444, China; yangyue0328@shu.edu.cn

* Correspondence: gushenshen@shu.edu.cn

† Current address: 99 Shangda Road, Shanghai 200444, China.

Received: 5 January 2020; Accepted: 20 February 2020; Published: 22 February 2020



Abstract: The Max-cut problem is a well-known combinatorial optimization problem, which has many real-world applications. However, the problem has been proven to be non-deterministic polynomial-hard (NP-hard), which means that exact solution algorithms are not suitable for large-scale situations, as it is too time-consuming to obtain a solution. Therefore, designing heuristic algorithms is a promising but challenging direction to effectively solve large-scale Max-cut problems. For this reason, we propose a unique method which combines a pointer network and two deep learning strategies (supervised learning and reinforcement learning) in this paper, in order to address this challenge. A pointer network is a sequence-to-sequence deep neural network, which can extract data features in a purely data-driven way to discover the hidden laws behind data. Combining the characteristics of the Max-cut problem, we designed the input and output mechanisms of the pointer network model, and we used supervised learning and reinforcement learning to train the model to evaluate the model performance. Through experiments, we illustrated that our model can be well applied to solve large-scale Max-cut problems. Our experimental results also revealed that the new method will further encourage broader exploration of deep neural network for large-scale combinatorial optimization problems.

Keywords: Max-cut problem; combinatorial optimization; deep learning; pointer network; supervised learning; reinforcement learning

1. Introduction

Combinatorial optimization is an important branch of operations research. It refers to solving problems of variable combinations by minimizing (or maximizing) an objective function under given constraints, and is based on the research of mathematical methods to find optimal arrangements, groupings, orderings, or screenings of discrete events. As a research hot-spot in combinatorial optimization, the Max-cut problem is one of the 21 typical non-deterministic polynomial (NP)-complete problems proposed by Richard M. Karp [1]. It refers to obtaining a maximum segmentation for a given directed graph, such that the sum of the weights across all edges of two cutsets is maximized [2]. The Max-cut problem has a wide range of applications in engineering problems, such as Very Large Scale Integration (VLSI) circuit design, statistical physics, image processing, and communications network design [3]. As a solution of the Max-cut problem can be used to measure the robustness of a network [4] and as a standard for network classification [5], it can also be applied to social networks.

It has been discovered that many classic combinatorial optimization problems derived from engineering, economics, and other fields are NP-hard. The Max-cut problem concerned in this paper

is among these problems. For combinatorial optimization problems, algorithms can be roughly divided into two categories: one is represented by exact solution approaches, including enumeration methods [6] and branch and bound methods [7], etc. The other category is represented by heuristic algorithms including genetic algorithms, ant colony algorithms, simulated annealing algorithms, neural networks, Lin–Kernighan Heuristic (LKH) algorithms, and so on [8]. However, there is no polynomial time solvable algorithm to find a global optimal solution for NP-hard problems. Compared with the exact approach, heuristic algorithms are able to deal with large-scale problems efficiently. They have certain advantages in computing efficiency and can be applied to solving large-scale problems with huge amount of variables. In order to solve the Max-cut problem, a large number of heuristic algorithms have been proposed, such as evolutionary algorithms and ant colony algorithms. However, for these algorithms, the most obvious disadvantage of them is that they are easy to fall into local optima. For this reason, more and more experts have begun working on the research and innovation of some novel and effective algorithms for large-scale Max-cut problems.

Deep learning is a research field which has developed very rapidly in recent years, achieving great success in many sub-fields of artificial intelligence. From its root, deep learning is a sub-problem of machine learning. Its main purpose is to automatically learn effective feature representations from a large amount of data, such that it can better solve the credit assignment problem (CAP) [9]; that is, the contribution or influence of different components in a system or their parameters to the output of the final system. The emergence of deep neural networks has provided the possibility for solving large-scale combinatorial optimization problems. In recent years, with the development of the combination of deep neural networks and operations research for large-scale combinatorial optimization problems, scholars have explored how to apply deep neural networks in these fields, and have achieved certain results. The related research has mainly focused on the algorithm design for combinatorial optimization problems based on pointer networks. Vinyals used the attention mechanism [10] to integrate a pointer structure into the sequence-to-sequence model, thus creating the pointer network. Bello improved the pointer network structure and used a strategy gradient algorithm combined with time-series difference learning to train the pointer network in reinforcement learning to solve the combinatorial optimization problem [11]. Mirhoseini removed the coding part of a recurrent neural network (RNN) and used the embedded input to replace the hidden state of the RNN. With this modification, the computational complexity was greatly reduced and the efficiency of the model was improved [12]. In Reference [13], a purely data-driven method to obtain approximate solutions of NP-hard problems was proposed. In Reference [14], a pointer network was used to establish a flight decision prediction model. Khalil solved classical combinatorial optimization problems by Q-learning [15]. The pointer network model has also been used, in Reference [16], to solve the weightless Max-cut problem. Similarly, solved the unconstrained boolean quadratic programming problem (UBQP) through the pointer network [17].

The section arrangement of this paper is as follows. Section 2 mainly introduces the Max-cut problem and the method for generating its benchmark. Section 3 demonstrates the pointer network model, including the Long Short-Term Memory network and Encoder–Decoder. Section 4 introduces two ways to train the pointer network model to solve the Max-cut problem, namely supervised learning and reinforcement learning. Section 5 illustrates the details of the experimental procedure and the results. Section 6 provides the conclusions.

2. Motivation and Data Set Structure

2.1. Unified Model of the Max-Cut Problem

The definition of the Max-cut problem is given as follows.

An undirected graph $G = (V, E)$ consists of a set of vertices V and a set of edges E , where $V = \{1, 2, \dots, n\}$ is its set of vertices, and $E \subseteq V \times V$ is its set of edges, and $w_{i,j}$ is the weight on the edge connecting vertex i and vertex j . For any proper subset S of the vertex set V , let:

$$\delta(S) = \{e_{i,j} \in E; i \in S, j \in V-S\}, \quad (1)$$

where $\delta(S)$ is a set of edges, one end of which belongs to S and the other end belongs to $V-S$. Then, the cut $cut(S)$ determined by S is:

$$cut(S) = \sum_{e_{i,j} \in \delta(S)} w_{i,j}. \quad (2)$$

In simple terms, the Max-cut problem is to find a segmentation $(S, V-S)$ of a vertex set V , where the maximum weight of the edges is segmented.

2.2. Benchmark Generator of the Max-Cut Problem

When applying deep learning to train and solve the Max-cut problem, whether supervised learning or reinforcement learning, a large number of training samples are necessary. The method of data set generation introduced here is to transform the $\{-1, 1\}$ quadratic programming problem into the Max-cut problem.

First of all, the benchmark generator method for the boolean quadratic programming (BQP) problem, proposed by Michael X. Zhou [18], is used to generate random $\{-1, 1\}$ quadratic programming problems, which can be solved in polynomial time. Next, inspired by [19], we transform the results of the previous step into solutions of the Max-cut problem. The specific implementation is described below.

Michael X. Zhou transformed the quadratic programming problem shown by Equation (3) into the dual problem shown by Equation (4) through the Lagrangian dual method.

$$\min \{f(x) = \frac{1}{2}x^T Qx - c^T x \mid x \in \{-1, 1\}^n\}, \quad (3)$$

where $Q = Q^T \in \mathbb{R}^{n \times n}$ is a given indefinite matrix, and $c \in \mathbb{R}^n$ is a given non-zero vector.

The dual problem is described as follows:

$$\begin{aligned} \text{find } & Q, c, x, \lambda \\ \text{s.t. } & (Q + \text{diag}(\lambda)) = c \\ & Q + \text{diag}(\lambda) > 0 \\ & x \in \{-1, 1\}^n \end{aligned} \quad (4)$$

Then, according to the paper [19], the solution of the $\{-1, 1\}$ quadratic programming problem can be transformed into the solution of the Max-cut problem.

The integer programming for the Max-cut problem is given by:

$$\begin{aligned} \max \quad & \frac{1}{2} \sum_{i < j} w_{i,j} (1 - x_i \cdot x_j) \\ \text{s.t. } \quad & x_i \in \{-1, 1\}, \forall i = 1, \dots, n, \end{aligned} \quad (5)$$

where i in $x_i \in \{-1, 1\}$ represents the vertex i , and -1 and 1 represent the values of the two sets. If $x_i \cdot x_j$ is equal to 1 and the vertices of edge (i, j) are in the same set, then $(i, j) \in E$ is not a cut edge; if $x_i \cdot x_j$ is equal to -1 and the vertices of the edge (i, j) are not in the same set, then $(i, j) \in E$ is the cut edge. If $(i, j) \in E$ is a cut edge, $(1 - x_i \cdot x_j)/2$ is equal to 1 ; if $(i, j) \in E$ is not a cut edge, $(1 - x_i \cdot x_j)/2$ is equal to 0 . Thus, the objective function represents the sum of the weights of the

cut edges of the Max-cut. Define $S = \{i : x_i = 1\}$, $\bar{S} = \{i : x_i = -1\}$, and the weight of the cut is $w(S, \bar{S}) = \sum_{i < j} w_{i,j}(1 - x_i \cdot x_j)/2$.

The pseudocode for generating the benchmark of the Max-cut problem is shown in Algorithm 1, where the parameter *base* is used to control the value range of the elements in matrix Q .

Algorithm 1 A benchmark generator for the Max-cut problem

Input: Dimension: n ; *base* = 10;

Output: Matrix: Q ; Vector: x

- 1: Randomly generate an n -dimensional matrix that conforms to the standard normal distribution to obtain Q ;
 - 2: $Q = \text{base} \times Q$;
 - 3: Convert Q to a symmetric matrix with $\frac{Q+Q^T}{2}$;
 - 4: Generate random numbers in the range (0,1) of n rows and 1 column as a vector x ;
 - 5: $x = 2x - 1$;
 - 6: Take the absolute value of Q and sum it over the rows, assigning the result to λ ;
 - 7: Place the value of the vector λ on the main diagonal of the square matrix Q' in order, and let the values of Q' (except the main diagonal) be zero.
 - 8: $c = (Q + Q') \times x$;
 - 9: Set an additional variable w_{oj} , $w_{oj} = \frac{1}{4}(\sum_{j=1}^{i-1} q_{ji} + \sum_{j=i+1}^n q_{ij}) + \frac{1}{2}c_i$, $1 \leq j \leq n$, and $w_{ij} = \frac{1}{4}q_{ij}$, $1 \leq i < j \leq n$;
 - 10: Update Q : $Q = (q_{1j}^T, q_{2j}^T, \dots, q_{(n+1)j}^T) \leftarrow (w_{0j}^T, w_{1j}^T, \dots, w_{nj}^T)$;
 - 11: Set an additional variable $x_0 = 1$, and let $x = 2x + 1$;
 - 12: Update x : $x = (x_1, x_2, \dots, x_{n+1}) \leftarrow (x_0, x_1, \dots, x_n)$.
-

This method for obtaining Max-cut benchmark data sets effectively solves the difficulty in training the network to solve the Max-cut problem model when lacking a large number of training samples. However, there is a common defect in this method: in the training set obtained using the dual problem to deduce the solution of the original problem, its data samples obey certain rules. This may lead to difficulty in learning the general rule of the Max-cut problem when training with the method by deep learning.

Therefore, in addition to the above method, we consider using the benchmark generator in the Biq Mac Library to solve the Max-cut problem. The Biq Mac Library offers a collection of Max-cut instances. Biq Mac is a branch and bound code based on semi-definite programming (SDP). The dimension of the problems (i.e., number of variables or number of vertices in the graph) ranges from 60–100. These instances are mainly used to test the pointer network model for the Max-cut problem.

3. Models

3.1. Long Short-Term Memory

It is difficult for traditional neural networks to classify subsequent events by using previous event information. However, an RNN can continuously operate information in a cyclic manner to ensure that the information persists, thereby effectively processing time-series data of any length. Given an input